
Can Computational Reducibility Lead to Transferable Models for Graph Combinatorial Optimization?

Semih Cantürk^{*1,2} Thomas Sabourin^{*3,2} Frederik Wenkel^{†4} Michael Perlmutter⁵ Guy Wolf^{3,2}

Abstract

A key challenge in developing unified neural solvers for combinatorial optimization (CO) is the efficient generalization of models from a given set of tasks to new tasks unseen during initial training. To address this, we first establish a new GNN encoder, which uses a GCON module as a form of expressive message passing together with energy-based unsupervised loss functions. This model achieves highly competitive performance across multiple CO tasks when trained individually on each task. We then leverage knowledge from the computational reducibility literature to propose pretraining and fine-tuning strategies that transfer effectively (a) between MVC, MIS and MaxClique, and (b) in a multi-task learning setting that additionally incorporates MaxCut, MDS and graph coloring. Additionally, in a leave-one-out, multi-task learning setting, we observe that pretraining on all but one task almost always leads to faster convergence on the remaining task when fine-tuning, while avoiding negative transfer. Our findings indicate that learning common representations across multiple graph CO problems is viable through the use of expressive message passing coupled with pretraining strategies that are informed by the polynomial reducibility literature, thereby taking an important step towards enabling the development of foundational models for neural CO. We provide an open source implementation of our work at <https://github.com/semihcanturk/COPT-MT>.

1. Introduction

The ability to transfer performance from one task to another is a crucial challenge in the development of modern AI since it eliminates the need to train a model from scratch each time that a new task is encountered. Instead, we seek to train a model to perform well on a representative collection of problems in such a way that it may be easily adapted to each new task in a relatively lightweight manner. Common techniques for this include partial fine-tuning as well as attaching relatively simple downstream task heads to an otherwise frozen architecture (Oquab et al., 2014; Zeiler & Fergus, 2014; Devlin et al., 2019; You et al., 2021; Kaur et al., 2021). For example, in medical image analysis, pretrained vision models have been shown to provide meaningful features, significantly improving conventional ML efficacy (Kaur & Mahajan, 2025), and a promising initialization for further fine-tuning under data scarcity conditions (Eliwa, 2025).

We note that while transferability has only recently emerged to the forefront of modern ML/AI, it has been a cornerstone of theoretical CS for many decades. Indeed, the study of computability and complexity relies on a hierarchical structure of equivalence classes defined by appropriate reductions between problems (Cormen et al., 2022; Papadimitriou & Steiglitz, 1998). Perhaps the most prominent examples are the classes P and NP, containing problems that are solvable in deterministic vs. non-deterministic polynomial time (correspondingly). To show that a problem is in P, one has to either find a direct algorithm that solves it, or – perhaps more often – an indirect solution via a polynomial *reduction* to another problem that is known to be in P. This reduction would translate the inputs to the target problem, and then translate the solution from it back to that of source problem. As long as both translation steps can be done in polynomial time, their cascade with the polynomial algorithm for the target problem would also result in a polynomial algorithm. Similarly, membership in NP can be established either by verifying a hypothesized solution in polynomial time, either directly, or indirectly via reduction to a known NP problem (i.e., by translating the candidate solution to test on it). In particular, the study of such reductions gives rise to the notion of NP-complete (or NP-hard) problems – those to which every problem in NP can be (polynomially) re-

^{*}Equal contribution [†]Work done while affiliated with ^{1,2}.

¹Dept. of Computer Science and Operations Research, Université de Montréal ²Mila – Quebec AI Institute ³Dept. of Mathematics & Statistics, Université de Montréal ⁴Valence Labs ⁵Dept. of Mathematics, Boise State University. Correspondence to: Semih Cantürk <semih.canturk@umontreal.ca>, Thomas Sabourin <thomas.sabourin@umontreal.ca>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

duced. Indeed, these problems form an upper bound on NP complexity, in the sense that if any of them is solvable in polynomial time, it would immediately enable polynomially solving any problem in NP via a cascade of reductions.

We note that while the discussion above focused on polynomial complexity, numerous other complexity classes have been extensively studied, such as poly-logarithmic, linear, or randomized-polynomial time, as well as ones focusing on other algorithmic aspects, such as parallelization capabilities and space requirements. A common theme in each of these is the reliance on complexity-bounded reductions, yielding the notion of completeness (or hardness) of problem classes. Further, in these studies, typically only a handful of core problems that are directly solved via a handcrafted algorithm, whereas the other problems are indirectly solved by reducibility. We refer the reader to [Cormen et al. \(2022\)](#); [Papadimitriou & Steiglitz \(1998\)](#) for further details and extensive background on these topics. Intuitively, one could argue that the aim of foundation models in deep learning is similar, at least in spirit, to these studies, in that they aim to establish a core set of tasks (with associated loss terms and curated data) that are sufficient to elicit emergent behavior, by which entire families of problems (or tasks) will be easily, if not trivially, approached with relatively simple adaptation of the core foundation model.

In this work, we investigate whether the notion of efficient (e.g., complexity-bounded) reducibility can inspire, or potentially inform, decisions regarding transferability. This is particularly relevant in combinatorial optimization problems on graphs, which have been of high interest in recent graph neural network studies, as well as in theoretical computer science, motivated in part by applications in, e.g., logistics ([Bao et al., 2018](#)), healthcare ([Zhong & Tang, 2021](#)), and scientific discovery ([Naseri & Koffas, 2020](#)).

Notably, most graph CO problems are NP-hard and feature an exponential search space, which makes their direct solution computationally prohibitive. Nevertheless, numerous recent work have produced promising results for generating approximate solutions for individual CO tasks, often using networks that are trained by task-specific loss functions ([Lucas, 2014](#); [Karalias & Loukas, 2020](#); [Min et al., 2022](#); [Zhang et al., 2023](#)). Here, we seek to build on their work towards a unified model that is able to solve multiple CO tasks simultaneously, utilizing connections between transferability and reducibility. In particular, we aim to replace traditional reductions (driven by complexity bounds) with efficient transfer learning mechanisms suitable for integration in deep learning paradigms, and establish empirically their cross-task generalization feasibility. In doing so, we aim to provide an important stepping stone towards the development of foundation models that form a universal neural solver for CO problems.

To this end, we first establish, in [Sec. 3](#), new baselines for individual CO tasks based on the Graph Combinatorial Optimization Network (GCON) network introduced by [Wenkel et al. \(2025\)](#) and show that this method matches the performance of most current methods, and occasionally challenges the state-of-the-art (albeit not our primary focus here). Namely, we focus on the following six problems, described in further detail in [Sec. 3.3](#): Maximum Independent Set (MIS), Minimum Dominating Set (MDS), Minimum Vertex Cover (MVC), Max Clique, Max Cut, and Graph Coloring. We then turn our attention to transferability, first studying in [Sec. 4](#) pairwise transferability between three of these tasks. There, we explore the relation between reducibility and transferability, while also elucidating the challenges in translating theoretical knowledge to practical application in deep learning settings. Next, in [Sec. 5](#) we shift towards a more realistic setting, addressing a primary prerequisite for the development of transferable foundation models, which are based on forming a unified trunk pre-trained via multi-task learning, and that can generalize to new tasks with relatively light-weight adaptation.

Our results indicate that while indeed some connections arise between the notions of reducibility and neural transferability, these connections are not trivial and further work is needed to fully elucidate them. Nevertheless, we see our contributions as providing a stepping stone for further studies on this topic, and a promising starting point for future work on universal neural CO solvers.

2. Related Work

Several prominent lines of research have emerged as part of the recent interest in graph CO problems. We focus here on a brief summary of the ones most relevant to this work.

Neural CO. The discrete nature of combinatorial optimization problems and their natural representability through graphs render them of particular interest to the graph learning community. As part of the recent interest in neural CO solvers, several prominent lines of research emerged. Most early prominent works, such as [Gasse et al. \(2019\)](#); [Li et al. \(2018\)](#) and [Selsam et al. \(2019\)](#), have focused on *supervised* approaches for neural CO. However, supervised approaches are inherently limited in that obtaining labels for NP-hard problems becomes computationally infeasible. Consequently, other lines of works have taken precedence in recent years, namely reinforcement learning (RL) ([Dai et al., 2017](#); [Bello et al., 2017](#); [Yolcu & Póczos, 2019](#)) (and related methods like generative flow networks ([Zhang et al., 2023](#))), and unsupervised learning ([Tönshoff et al., 2021; 2023](#); [Amizadeh et al., 2019; 2020](#); [Sanokowski et al., 2024](#)). Unsupervised methods are typically concerned with designing task-specific loss functions such that the optimal solution to the CO problem minimizes this loss.

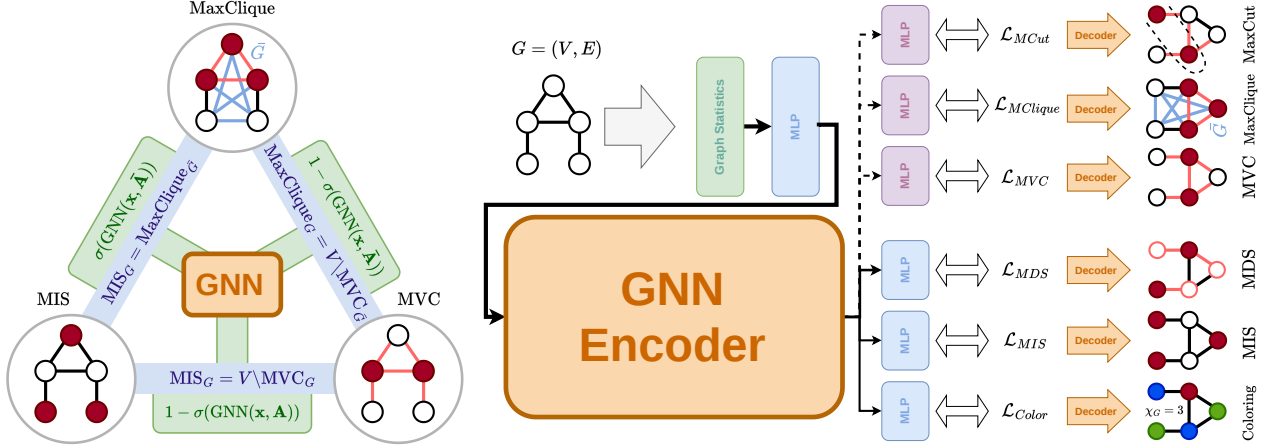


Figure 1. (Left) Reduction between tasks for pairwise transferability section: MIS/MVC are complements, but MaxClique is based on an auxiliary graph. (Right) Multi-task learning / fine-tuning architecture (pretraining set in green, fine-tuning set in purple).

Our work is closely related to a family of methods that train graph neural networks (GNN) in an unsupervised, end-to-end differentiable manner. Karalias & Loukas (2020) spearheaded this direction by proposing a surrogate loss function for the MaxClique and graph partitioning problems, together with sequential decoding of probabilistic predictions to obtain valid solutions. Min et al. (2022) paired this framework with a GNN derived from the geometric scattering transform (Gao et al., 2019) to obtain further competitive results on approximate MaxClique, while Min et al. (2023) used a similar setup for the traveling salesman problem. A recent subset of unsupervised GNN-based solvers use *physics-inspired* loss functions based on Lucas (2014), which show that many CO problems map to the Ising model, and minimizing the corresponding Hamiltonian representing its energy landscape is equivalent to solving them. Schuetz et al. (2022a); Krylova & Phillipson (2025) tackle MaxCut and MIS using this recipe while Schuetz et al. (2022b) extend it to graph coloring. More recently, Sanokowski et al. (2024) used task-specific Hamiltonians to train a diffusion model and attain competitive results across multiple tasks.

Multi-Task neural CO. Most attempts to develop a unified model for CO problems are based on supervised or reinforcement learning. Examples include GOAL (Drakulic et al., 2024), a generalist model consisting of a single backbone based on mixed-attention blocks that learns from imitating expert trajectories, UniCO (Zong et al., 2025), framing problems as Markov Decision Process and incorporating a CO-prefix design and a two-stage self-supervised learning scheme, and the RL- models in Li et al. (2025); Wang et al. (2025), based on gradient homogenization and a multi-armed bandits respectively. Other methods (Berto et al., 2024; Lin et al., 2024; Liu et al., 2024; Zhou et al., 2024) propose RL-based pretraining-fine-tuning approaches

specifically for different variations of vehicle routing problems (VRCs). Finally, alternative approaches aim to obtain generic representations via SAT bipartite graphs (Guo et al., 2025; Zeng et al., 2023) or syntax trees capturing problem constraints (Boisvert et al., 2024). Unlike these methods, we rely on unsupervised learning with task-specific loss functions, which avoids problems arising from reliance on labels or the large size of the state space in RL. Moreover, we avoid transforming the graphs to a common SAT or abstract representation but rather work on the original graph structure (see Appendix C for a comparison of the two approaches) by using a GCON backbone that leverages a complex filter bank and localized attention mechanisms to learn approximate solutions to CO problems (Wenkel et al., 2025). Finally, unlike most of these methods, which do not justify their selection of pretraining tasks theoretically, we aim to use known polynomial reductions to show similar problems transfer effectively between each other, thus guiding our pretraining tasks selection.

Polynomial reductions for CO problems. Combinatorial optimization problems have been extensively studied in the computer science literature, for example with Karp’s 21 NP-complete problems (Karp, 1972). Many polynomial reductions are well known between these problems. For instance, reductions between the maximum clique (MaxClique), maximum independent set (MIS) and minimum vertex cover (MVC) can be obtained from the following lemma.

Lemma 2.1 (Garey & Johnson, 1979). *For any graph $G = (V, E)$ and subset $V' \subseteq V$, the following statements are equivalent: 1. V' is a vertex cover for G ; 2. $V \setminus V'$ is an independent set for G ; and 3. $V \setminus V'$ is a clique in the complement $\bar{G}$ of G , where $\bar{G} = (V, \bar{E})$.*

Moreover, Filar et al. (2019) demonstrate the existence of a kernel subset of Karp’s 21 problems. Specifically, they define problem Q as belonging to the linear orbit of P if there exists a reduction where the input size of Q is bounded by a linear function of the input size of P . They then show that every problem in Karp’s original set can be reduced with linear growth in problem size to at least one problem within this kernel. In particular, their work implies that MaxClique, MaxCut, Node Cover (MVC and therefore MIS), and Set Cover (of which MDS is a special case) reside in the linear orbit of 0-1 Programming. Note that conversely, problems like Chromatic Number (K -coloring) and the Hamiltonian Cycle Problem appear to occupy different linear orbits.

3. Methodology

Table 1. Energy-based loss functions for CO problems considered. Table extended from Sanokowski et al. (2024).

CO PROBLEM	OBJECTIVE: $\min_{X \in \{0,1\}^N} H(X)$
MIS	$H(X) = -A \sum_{i=1}^N X_i + B \sum_{(i,j) \in \mathcal{E}} X_i \cdot X_j$
MDS	$H(X) = A \sum_{i=1}^N X_i + B \sum_{i=1}^N (1 - X_i) \prod_{j \in \mathcal{N}(i)} (1 - X_j)$
MAXCLIQUE	$H(X) = -A \sum_{i=1}^N X_i + B \sum_{(i,j) \notin \mathcal{E}} X_i \cdot X_j$
MAXCUT	$H(\sigma) = -\sum_{(i,j) \in \mathcal{E}} \frac{1 - \sigma_i \sigma_j}{2}$ WHERE $\sigma_i = 2X_i - 1$
MVC	$H(X) = A \sum_{i=1}^N X_i + B \sum_{(i,j) \in \mathcal{E}} (1 - X_i) \cdot (1 - X_j)$
K -COLORING	$H(X) = \sum_{i=1}^N (1 - \sum_{k=1}^K X_{i,k})^2 + \sum_{(i,j) \in \mathcal{E}} \sum_{k=1}^K X_{i,k} X_{j,k}$

We frame each CO task as an unsupervised learning problem, and assume we do not have access to the set of interest or even the size of this set. Throughout, we assume undirected and unweighted graphs $G := (V, E)$, $V = \{v_1, \dots, v_N\}$, though much of our work can easily be extended to weighted graphs. We denote the complement of G as $\bar{G} := (V, \bar{E})$.

When training models from scratch, our pipeline largely follows Wenkel et al. (2025) and Karalias & Loukas (2020). We first train a GNN encoder, which is trained by a task-specific, unsupervised loss function to generate a vector $\mathbf{p}$ whose i -th entry $\mathbf{p}_i = \mathbf{p}[v_i]$ is interpreted as the probability that v_i is in the set of interest. That is, the $\mathbf{p}_i$ may be thought of a soft proxies for the indicator variables $X_i \in \{0, 1\}$ which indicate whether v_i is a member of this set. We then use a sequential, rule-based decoder that strictly enforces the problem constraints to arrive at the final set of interest. We discuss the individual elements of our pipeline below.

We note that throughout this work, we will frequently use the terms reducibility and transferability. By reducibility, we refer to the existence of a polynomial-time reduction between computational problems in the standard sense of theoretical computer science, as used in complexity theory to relate decision or optimization problems. By transferability, we refer to the machine learning paradigm in which knowledge acquired from source tasks is leveraged to improve performance on a related target task. This can be

achieved through various mechanisms, including partial parameter reuse (e.g., adapting only task-specific components such as output heads) or full fine-tuning of all model parameters from a pretrained initialization.

3.1. GNN Encoder

Our GNN encoder relies on the Graph Combinatorial Optimization Network (GCON) architecture introduced in Wenkel et al. (2025). Unlike local message-passing GNNs such as GCN, GIN or GAT which effectively perform low-pass filtering over the graph, GCON uses a rich bank of multi-scale wavelet filters inspired by the geometric scattering transform (Gao et al., 2019; Gama et al., 2018; Zou & Lerman, 2020; Perlmutter et al., 2023; Chew et al., 2024). Wenkel et al. (2025) demonstrate that such networks can learn rich node representations, and avoid typical information bottlenecks of localized message-passing.

We follow a a common architectural design of a linear layer before and after the GNN layers, as well as GELU (Hendrycks & Gimpel, 2016) nonlinearities, batch normalization (Ioffe & Szegedy, 2015), and graph size normalization after each layer. We additionally concatenate all GNN layer outputs as our input to the final linear layer, which empirically helps performance over most tasks. We apply sigmoid activation after the final layer to map the outputs to $[0, 1]$.

As in Wenkel et al. (2025), we use node degrees, local clustering coefficients, and triangle counts as node features. Empirically, this performed superior compared to the Dirac encodings used in Karalias & Loukas (2020) and largely on par with Laplacian and random walk-based encodings. The output of our GNN is a vector $\mathbf{p}$ where $\mathbf{p}_i$ denotes the probability that v_i is in the set of interest, with the exception of K -coloring, for which we output K vectors where $(\mathbf{p}_k)_i$ is thought of as the probability that v_i has color k .

Table 2. Performance comparison of GCON with other baselines on **RB-small** datasets (Mean $\pm$ Std). The best deep learning-based method is listed in **gold**, second best in **silver**. † indicates Gurobi outperforms all deep learning methods.

METHOD	TYPE	MVC $\downarrow$	MCLIQUE $\uparrow$	MIS $\uparrow$
TRUE SIZE	—	206.95	19.07	20.07
GUROBI	OR	—	19.05 †	19.98 †
GFN	SSL	—	16.24	19.18
GCN	SSL-GNN	221.56 $\pm$ 0.05	15.33 $\pm$ 0.04	17.67 $\pm$ 0.20
GIN	SSL-GNN	221.63 $\pm$ 0.23	15.28 $\pm$ 0.14	17.50 $\pm$ 0.04
GATv2	SSL-GNN	220.76 $\pm$ 2.26	15.56 $\pm$ 0.09	17.58 $\pm$ 0.07
GCON	SSL-GNN	211.69 $\pm$ 0.16	16.92 $\pm$ 0.13	18.12 $\pm$ 0.11

3.2. Sequential Decoder

The sequential decoder processes the probabilistic output $\mathbf{p}$ of the GNN encoder and enforces the hard problem con-

straints to arrive at a valid solution. In each case, we order the nodes by probability of being included in the solution set (based on $\mathbf{p}$) and initialize the solution set $S^* = \emptyset$. We then iterate over the ordered node set and add v_i to S^* only if it does not violate the problem constraints.

As demonstrated in Karalias & Loukas (2020), this blueprint follows the method of conditional expectation (Raghavan, 1988). However, we note that this approach does not guarantee the optimal solution, especially when the GNN encoder assigns similar high probabilities to multiple nodes that may constitute distinct solution sets. Therefore, we employ k seeds where each seed selects a different node to initialize the set with (choosing the k nodes with the highest probabilities as initial nodes). We construct k sets, $S_1^*, \dots, S_k^*$, and return the largest or smallest depending on the task. We note that while a similar approach is proposed in Wenkel et al. (2025), their implementation constructed the k sets sequentially, leading to dramatically increased runtimes. By contrast, our sets are constructed in a parallelized manner, enabling the efficient use of higher k . We give more details about the decoders in Appendix B.

3.3. Objective Functions

As objective functions, we rely on the Ising formulations of the CO problems, as provided in Lucas (2014) and displayed in Table 1. When training our network, we optimize the probabilistic vector $\mathbf{p}$ rather than $X \in \{0, 1\}^N$. The Ising model and equivalent quadratic unconstrained binary optimization (QUBO) formulations have emerged as an effective framework to unify many CO problems by assigning an energy function (namely the Hamiltonian) to every task such that its optimal solution minimizes the total energy (Lucas, 2014; Kochenberger et al., 2014; Glover et al., 2022; Schuetz et al., 2022a). The loss functions for MIS, MDS, MaxClique and MDS involve two terms A and B that can be tuned additionally; the ratio of A to B controls how strongly to penalize constraint violations in relation to maximizing or minimizing the size of the objective set. Setting $A < B$ ensures the optimal solutions are valid subsets. We note that all six problems studied here (illustrated in Fig. 1) are well established in relevant computer science literature, and their approximate relaxations used to derive the loss terms in Table 1 are becoming increasingly standard in GNN literature. Detailed problem descriptions are provided in Appendix A.

3.4. Ablation Study on GCON Layer

To validate our use of GCON in the following studies, we first present an ablation study comparing several GNN-based neural solvers with identical architectures (except the convolutional layer), as well as the performant generative flow network-based GFN model (Zhang et al., 2023) on MVC, MaxClique and MIS. As shown in Table 2, when

paired with the energy-based loss functions, we outperform all deep learning baselines (with the exception of GFN on MIS, which is very competitive), attaining particularly strong results for MVC and MaxClique on RB-small graphs. The best-performing GCON models also constitute our pre-trained models for our transferability experiments in Sec. 4.

We highlight that, for the MaxClique problem on RB-small, we substantially improve over Wenkel et al. (2025), finding a MaxClique size of 16.92 (compared to 15.87 in Wenkel et al. (2025)) despite the fact that they used a nearly identical architecture. This difference is attributed to our improved loss function that follows the Hamiltonian: By contrast, Wenkel et al. (2025) follows a formulation based on the Motzkin-Strauss theorem (Motzkin & Straus, 1965), which – while also valid and related – uses a quadratic first term that changes the optimization landscape, likely over-emphasizing size maximization over the penalty term. To verify our observations, we further tested a similar quadratic first term for MIS (note the similarity of the MIS and MaxClique Hamiltonian losses in Table 1); the alternative loss only attained 15.5 average MIS size as opposed to 18.04 with the Hamiltonian loss. These findings strongly emphasize the importance of an appropriate loss function selection, despite the availability of many valid formulations.

4. Pairwise Transferability

We begin this section with a brief discussion of known polynomial reductions among three closely related CO tasks: Maximum Independent Set (MIS), Minimum Vertex Cover (MVC) and Maximum Clique (MaxClique).

- **MIS $\leftrightarrow$ MVC:** MIS and MVC are trivially reducible as they are the complements of each other by Lemma 2.1: Solving for the MIS subset $\text{MIS}_G \subset V$ implies $\text{MVC}_G = V \setminus \text{MIS}_G$, and vice versa.
- **MaxClique $\leftrightarrow$ MIS:** Again by Lemma 2.1, the MaxClique of G is the MIS of the complement graph $\text{MaxClique}_G = \text{MIS}_{\bar{G}}$.
- **MaxClique $\leftrightarrow$ MVC:** Following the statements above, the MaxClique of G is the complement set of the MVC of the complement graph $\bar{G}$: $\text{MaxClique}_G = V \setminus \text{MVC}_{\bar{G}}$.

4.1. MIS $\leftrightarrow$ MVC

As established above, MIS_G and MVC_G are complements, and therefore solve each other implicitly. In our training pipeline, we can interpret this in terms of the node probabilities, where for a given node v_i , we have $\mathbf{p}_i^{\text{MIS}} = 1 - \mathbf{p}_i^{\text{MVC}}$.

We conjecture that the graph representations learned on MIS should also be sufficient to solve MVC, and vice versa. Furthermore, given the linear relationship between the two, a simple linear layer after message-passing should be able

to learn this function using identical representations. Our first test is thus to pretrain a model on one task, and transfer the representations to the other task using the pretrained GNN backbone and resetting & training only the linear post-message-passing layer; we expect the transferred model to quickly recover the original pretrained model performance.

RESULTS & ANALYSIS

Our hypothesis holds true with some caveats, with the results denoted in Table 3. We test several settings, the most straightforward of which is freezing the MIS-trained GNN backbone and simply resetting & fine-tuning the linear output layer to solve MVC, and vice versa. The linear model converges quickly (within 5 epochs for MIS, ~ 50 epochs for MVC), but does not match the baselines trained from scratch. This is likely due to the approximate nature of our solution, as the strict duality between the problems assumes an exact solution. In the second setting, instead of initializing the linear output layer from scratch, we invert it, i.e., we initialize the new linear layer by multiplying all parameters by -1 , as the inversion is maintained through the softmax output: $\sigma(-x) = 1 - \sigma(x)$. This provides a much better initialization as convergence is almost immediate on both tasks, but it does not lead to any substantial improvement on final results – implying that the frozen GNN representations themselves are not able to overcome the duality gap, and thus the backbone also needs to be fine-tuned.

Finally, we verify whether fine-tuning the whole model after inverting the output head can recover the baseline performance. For MIS $\rightarrow$ MVC in particular, the results are remarkable in that all runs converge within 15 epochs, and the resulting models outperform the baseline trained from scratch for 300 epochs. MVC $\rightarrow$ MIS is also successful, albeit to a lesser extent – likely because pretrained MIS model has converged to a better minimum: The fine-tuned model converges to a marginally worse performance than the baseline on average, and convergence takes 100-150 epochs, whereas the baselines were trained for 200 epochs. This indicates that the GNNs *can* learn to perform the appropriate reduction, but rely on good initialization and fine-tuning of the backbone to maximize performance – theoretical equivariance of the reduction alone is not enough.

Table 3. MIS $\leftrightarrow$ MVC transferability on **RB-small**. BASELINE refers to models trained from scratch on the CO task tested on. Other models are pretrained on the opposite task, e.g., the MIS column evaluates models pretrained on MVC and fine-tuned on MIS.

GNN	OUT-HEAD	MIS $\uparrow$	MVC $\downarrow$
BASELINE		18.12 $\pm$ 0.11	211.69 $\pm$ 0.16
FREEZE	RESET + FT	17.68 $\pm$ 0.04	212.46 $\pm$ 0.25
FREEZE	INVERT + FT	17.69 $\pm$ 0.05	212.39 $\pm$ 0.21
FT	INVERT + FT	18.00 $\pm$ 0.05	211.56 $\pm$ 0.24

4.2. MIS/MVC $\leftrightarrow$ MaxClique

Our experiments in 4.1 (MIS $\leftrightarrow$ MVC) implicitly use the fact that the problem reductions considered do not alter the graph topology. This increases transferability substantially as the graph representations do not suffer from any distribution shift in the graph structure nor the node features, which makes it viable to freeze the GNN backbone, and fine-tuning only the MLP head, in order to obtain close-to-optimal results. Indeed, the learned node-level representations in this case are sufficient, and no additional graph-level information is needed to solve either task.

However, this advantage does not apply to transferring representations between MaxClique and MIS/MVC: G and the complement graph $\bar{G}$ may have drastically different structural properties and distributions despite sharing the same node set V . For instance, RB graphs are typically sparse, which implies that their complements are very dense. This renders the structurally-derived node features (or any structurally-informed PSE that may be used in their stead for node identifiability) ineffective. Moreover, the graph convolutional layers will similarly suffer from this distribution shift.

With this in mind, we aim to answer the following questions on MIS $\rightarrow$ MaxClique transferability, while noting that our findings apply to MVC $\leftrightarrow$ MaxClique as well:

- **Exps. #4–7:** Despite the shift in graph topology, how useful are the MIS-pretrained weights by themselves in a (i) frozen backbone, or (ii) full fine-tuning setting?
- **Exps. #8–9:** Assuming the node-level representations are not sufficient, does additional *global* message-passing (using a stack of Graph Transformer (GT) layers on top of the GNN backbone) help adapt to the distribution shift in topology?
- **Exps. #10–11:** Recalling $\text{MaxClique}_G = \text{MIS}_{\bar{G}}$, can we match baseline performance by implementing the *true* reduction by fine-tuning the model to solve MIS on the complement $\bar{G}$? To account for the distribution shift in node features, we pretrain and fine-tune the model based on node features derived from *both* G and $\bar{G}$.

RESULTS & ANALYSIS

In Table 4, we compare our fine-tuned models with the baselines that comprise the first three rows: (1) GCON trained from scratch on MaxClique, using as node features graph statistics derived from G only and representing a typical training setting, (2) GCON similarly trained from scratch but node features derived from both G and $\bar{G}$ for fair comparison with fine-tuning settings that use $\bar{G}$, and (3) Randomly initialized GCON to benchmark random embedding performance. The baselines are trained for up to 700 epochs, while the frozen/fine-tuned models are trained for up to 200,

Table 4. Overview of MIS $\rightarrow$ MaxClique transferability experiments on **RB-small**. First two rows represent baselines trained from scratch. The “feats” column denotes whether graph statistics from only the original graph G or also its complement $\bar{G}$ are provided as initial node features. Third row represents a randomly initialized GCON model. Mean $\pm$ standard dev for 3 runs reported. Top 3 transfer runs are denoted with **gold**, **silver** and **bronze**.

#	FEATS	GNN STACK	GT	COMP?	MC SIZE
1	G	BASELINE	—	FALSE	16.92 ± 0.13
2	$G \parallel \bar{G}$	BASELINE	—	FALSE	16.63 ± 0.05
3	G	RANDOM	—	FALSE	10.71 ± 0.14
4	G	FROZEN	—	FALSE	16.12 ± 0.20
5	G	FINE-TUNED	—	FALSE	16.55 ± 0.03
6	$G \parallel \bar{G}$	FROZEN	—	FALSE	15.98 ± 0.12
7	$G \parallel \bar{G}$	FINE-TUNED	—	FALSE	16.63 ± 0.03
8	G	FROZEN	3-MHA	FALSE	16.13 ± 0.17
9	$G \parallel \bar{G}$	FROZEN	3-MHA	FALSE	16.11 ± 0.63
10	$G \parallel \bar{G}$	FROZEN	—	TRUE	15.52 ± 0.08
11	$G \parallel \bar{G}$	FINE-TUNED	—	TRUE	16.82 ± 0.04

with early stopping used.

All fine-tuning settings perform well above random, which is an encouraging if unsurprising result. We also note that even the worst-performing models are able to outperform local-message-passing GNNs trained from scratch (Table 2) and find an average maximum clique of size 16 (with the exception of #10). These results convincingly demonstrate that pretraining on MIS is *useful* for MaxClique even though the underlying topology of the MaxClique-reduced-to-MIS graphs are different. We also note that the flexibility of GCON layers likely contributes to the transferability of representations. Nevertheless, most fine-tuning settings fall short of the baselines. We hereby list our main takeaways on each question previously stated:

Freezing vs. Fine-tuning (#4–7): The aforementioned distribution shift problems limit the ceiling of the frozen-GNN-backbone models in the absence of additional learnable GNN layers, verifying that the MIS-trained local representations are not sufficient to predict MaxClique. Fine-tuning fares considerably better – while not always sufficient to match the baseline performance with only 200 epochs, #7 exactly matches its corresponding baseline (#2). While more topology-preserving reductions imply easier task transfer (as in the MIS $\leftrightarrow$ MVC case), these experiments show that CO tasks related via non-topology-preserving reductions *can* still provide a strong initialization, with the caveat that fine-tuning is required to close the distribution shift.

Global message-passing (#8–9): When considering additional global message-passing (i.e., graph Transformer) we focus on the frozen-backbone setting, since the fine-tuning setting renders them somewhat redundant, especially considering the additional quadratic computational complexity of the GT layers. We use three layers of multi-head attention with three heads each. We also evaluated combining MHA with message-passing using GraphGPS layers (Rampásek

et al., 2022), but did not see any additional benefits.

On average, global message-passing over frozen GNN backbones only offers marginal improvements, and falls short of trained baselines. However, we note high variance across runs, with one #9 run attaining 16.75, which outperforms the corresponding baseline (#2). This indicates that global message-passing *may* leverage the frozen MIS representations successfully to transfer to MaxClique, but naïve training of the transformer layers is likely insufficient – extensive fine-tuning and auxiliary structures (e.g., re-introducing node statistics or positional encodings, at the transformer-stack level, or deeper stacks with linear transformer layers) are required to obtain consistent improvements.

Implementing the true reduction (#10–11): Finally, we test the following setting: After pretraining our model to solve MIS, in the fine-tuning phase we perform message-passing over the complement graph $\bar{G}$ instead of the original. One can then use the MIS loss and metrics, effectively solving MIS over $\bar{G}$ which is equivalent to solving MaxClique (per the preamble of Sec. 4); note that keeping the MaxClique loss and metrics over the original graph is also viable as the losses are equivalent over the complement (Table 1).

Interestingly, if the MIS-pretrained backbone is frozen (#12), we obtain by far the worst-performing model, whereas fine-tuning the full model instead (#13) gives us the only model that can consistently match (and even beat) its baseline counterpart within the allotted epochs. The failure of the frozen model can likely be attributed to the distribution shift in the graph topology; however, the learned representations clearly form a useful initialization – fine-tuning quickly adapts the model to the new distribution to recover the baseline performance in less than a third of the epochs.

5. Multi-Task Transferability

We now study transferability between graph CO problems in a pretraining-fine-tuning framework, where we learn multiple CO problems simultaneously in pretraining and use this knowledge to transfer efficiently onto new problems by fine-tuning for a few epochs. We first show that a task only requires one task similar to itself in the pretraining set to learn a solution quickly when fine-tuned. We further show that almost all tasks benefit from fine-tuning on other tasks compared to training from scratch, and use our observations to partition the six tasks we consider into a pretraining set and a fine-tuning set of three tasks each. We then evaluate the resulting performances against single task learning baselines. We hope that a careful study of the interplay between different CO problems and known reductions will yield an appropriate pretraining set, paving the way to foundational models for graph CO problems. For every experiment in this section, we pretrain for 200 epochs on a Barabási–Albert

(BA)-small graph dataset and fine-tune on the same dataset for only 20 epochs. The model consists of a GCON backbone where we attach a simple MLP heads for every tasks, and the output is fed to a task specific unsupervised loss function (see Sec. 3.3 and Appendix A). Fine-tuning is done by attaching a new task head with the corresponding loss function and training for 20 epochs with either a frozen or an unfrozen backbone. The tasks considered here are MaxCut, MaxClique, MDS, MIS, MVC and K -coloring with $K = 10$. Tables display the size of the subset V' learned for all tasks but K -coloring, where we simply count the number of violations.

5.1. Leave-One-Out Transfer

We begin by asking if the successful transfers observed in Sec. 4 also apply in the multi-task learning setting. Suppose we want to fine-tune task T_n on a backbone pretrained on tasks $T_1, \dots, T_{n-1}$. To get efficient transfer, does it suffice for there to exist one task T_i in the pretraining set such that T_n is reducible to T_i ? To study this question, we look at MIS and MVC when they are fine-tuned on the other task (MIS from MVC and MVC from MIS), when they are fine-tuned on the rest of the tasks (MaxCut, MaxClique, 10-Coloring and MDS), and when they are fine-tuned on all tasks (rest + other). We compare the results to training MIS and MVC individually for 20 epochs, and show the results for a frozen and an unfrozen backbone in Figure 2.

Unsurprisingly, we find that when we transfer from MVC to MIS and vice-versa, we quickly converge to a good solution (within 4-6 epochs), whereas training from scratch takes at least 15 epochs to match the fine-tuning performance. However, if we do not freeze the backbone, we find that the behavior of the MIS learning curve when pretrained on all other tasks *including* MVC behaves similarly to when *only* pretrained on MVC, whereas when we remove MVC from the pretraining set, the curves behave closer to the training from scratch. Moreover, when the backbone is frozen, the model is not able to transfer to MIS using the other tasks if MVC is not present in the pretraining set, giving poor results, while if MVC is present, it actually converges faster than training from scratch. As expected, the inverse relation also holds true when transferring to MVC while keeping or removing MIS in the pretraining set. These results suggest that it is sufficient to have one $j \in \{1, \dots, n-1\}$ such that there is an efficient reduction from T_n to T_j in order to observe rapid transfer from the pretraining set to T_n that beats the baseline in a low-resource regime, even with a frozen backbone (thus requiring to train only a fraction of the total number of parameters).

Now we want set up a leave-one-out transfer framework to study if fine-tuning on a multi-task learning model pretrained on the five other tasks help the remaining task to

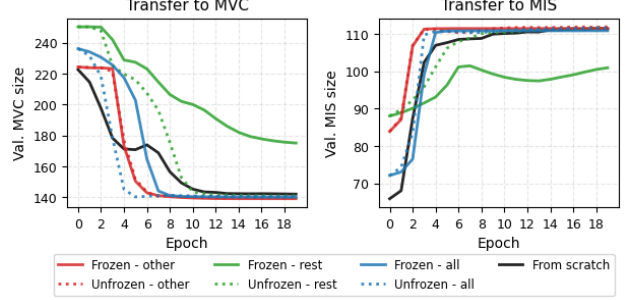


Figure 2. Transfer performance on MVC and MIS. We compare training from scratch against frozen and unfrozen fine-tuning on the **other** task (MVC on MIS and MIS on MVC), the **rest** of tasks (MCut, MClique, MDS, K -coloring), and **all** tasks (rest + other) for **BA-small** graphs.

Table 5. Leave-one-out fine-tuning in low-resource regime on **BA-small** graphs (20 epochs, unfrozen backbone, 5 runs average)

TASK	FROM SCRATCH	FINE-TUNED
↑ MAXCUT	716.81 ± 3.00	722.40 ± 1.17
↑ MAXCLIQUE	4.31 ± 0.01	4.32 ± 0.01
↓ MDS	35.57 ± 2.41	36.15 ± 1.87
↑ MIS	111.33 ± 0.20	111.56 ± 0.10
↓ MVC	141.30 ± 0.27	140.04 ± 0.33
↓ COLOR	61.92 ± 36.11	24.19 ± 9.08

learn a good solution faster than from scratch. We repeat this experiment five times for all tasks on BA small graphs in low-resources regime (20 epochs), and display the results in Table 5. We find that for every task but MDS, the model attains better performance at 20 epochs when it is fine-tuned on the other tasks than from scratch, which provides a strong evidence of transfer between graph CO problems. However, the benefits of fine-tuning varies between tasks: it is most beneficial for MaxCut and K -coloring to be fine-tuned on other CO problems, whereas MaxClique and MDS show little to no benefit. Therefore if our goal is to rapidly obtain good solutions to CO problems by fine-tuning tasks on a pretrained backbone compared to from scratch, our results suggest that MaxCut and K -coloring are good candidates to be fine-tuned.

5.2. Fine-Tuning on a Graph CO Backbone

In this section, we propose a pretraining set of three CO problems and leave the remaining three tasks to be fine-tuned for 20 epochs and compare performance to single task learning from scratch for 20 epochs and for the full 200 epochs. From our findings in in Sec. 5.1, we assert that having one closely related task in the pretraining set is sufficient for good fine-tuning performance. Since we want to manage our resources efficiently, we want to avoid having similar tasks in the pretraining set when only one is sufficient to learn the others by fine-tuning. Therefore,

Table 6. MDS ($\downarrow$) trained together with all but one task on **BA-small** graphs for 200 epochs (5 runs average)

MAXCUT	MAXCLIQUE	MIS	MVC	COLOR
33.36	41.34	41.40	34.68	45.29

Table 7. Fine-tuning on MDS-MIS-Coloring pretrained backbones for 20 epochs (**BA-small** dataset, 3 runs average)

TASK	FINE-TUNED	BASELINE	FULL
$\uparrow$ MAXCUT	722.77 ± 1.00	716.90 ± 1.55	726.58 ± 0.51
$\uparrow$ MAXCLIQUE	4.32 ± 0.01	4.31 ± 0.00	4.43 ± 0.02
$\downarrow$ MDS	29.65 ± 0.09	33.93 ± 1.15	29.56 ± 0.22
$\uparrow$ MIS	111.94 ± 0.06	111.33 ± 0.26	112.23 ± 0.06
$\downarrow$ MVC	139.70 ± 0.31	141.43 ± 0.23	139.40 ± 0.20
$\downarrow$ COLORING	17.29 ± 1.97	49.04 ± 21.84	3.52 ± 1.52

results from Sec. 4 suggest including only one of MIS, MVC and MaxClique in the pretraining set (preferably MIS or MVC), leaving one spot for MDS, K -coloring or MaxCut in the fine-tuning only set. Since MDS does not show any gains when fine-tuned on other tasks, unlike MaxCut and K -coloring, we choose to include it in the pretraining set. Moreover, we observe that when pretraining the all-but-one multi-task models, MDS performance is best when MaxCut and MVC are not pretrained alongside it, as we show in Table 6, suggesting to leave MaxCut and MVC out of the pretraining set. This implies by our previous remark that we will have MIS in pretraining and MaxClique in fine-tuning-only. We then delegate K -coloring to the pretraining set, which is also sensible since it is the most different from the other tasks, being the only one not linearly reducible to 0-1 programming as per Filar et al. (2019) (also see Sec. 2). In fact, we want to maximize task diversity in the pretraining set, so that the network has access to a wide range of information when fine-tuning. Therefore, we select MDS, MIS, K -coloring for pretraining and leave MaxClique, MaxCut, MVC to be fine-tuned. In Table 7 we show the results of fine-tuning all tasks on our backbone, which is unfrozen (except for fine-tuning K -coloring), and baseline results from training single tasks from scratch for 20 epochs (BASELINE) and 200 epochs (FULL).

We see that leveraging known polynomial reductions and interactions between CO tasks allows to carefully select a backbone of three tasks, to which we can attach simple task-specific MLP heads and fine-tune to get results that are on par with the fully trained (200 epochs) single task models for most tasks and that beat the low compute resources (20 epochs) single task models for all tasks. These results suggest that using reductions and interactions between tasks as a guide allows to select an efficient and transferable backbone, which becomes crucial as we increase the number of CO tasks. Therefore, we believe our methodology establishes an important step to building a foundational model

for combinatorial optimization problems on graphs.

6. Conclusion & Future Work

In this work, we have established a conceptual relation between computational reducibility – a topic traditionally associated with theoretical computer science, and learned transferability – an increasingly prominent area of machine learning in general, and deep learning in particular. We demonstrate that established knowledge of reducibility between problems can help identify and inform pretraining and transfer learning targets, although this relationship is not trivial, and further work remains to be done to fully understand the potential interaction between these notions. After initially exploring pairwise task transferability, we turned our attention to multi-task learning, which would be crucial for the formation of universal models (e.g., implemented via foundation models). We show that, indeed, this is a promising new direction for establishing theory-informed foundation models for graph CO, in which one may find a sufficient suite of landmark tasks that are sufficient for allowing efficient (if not ready-made) transfer of pretrained models to a wide family of significant tasks of interest.

Acknowledgements and Disclosure of Funding

This work was partially funded by Bourse en intelligence artificielle des Études supérieures et postdoctorales (ESP) 2023-2024 [Semih Cantürk]; the Fin-ML CREATE graduate studies scholarship for PhD, the J.A. DeSève scholarship for PhD and Guy Wolf’s research funds [Frederik Wenkel]; NSF OIA 2242769 [Michael Perlmuter]; Canada CIFAR AI Chair, IVADO (Institut de valorisation des données) grant PRF-2019-3583139727, FRQNT (Fonds de recherche du Québec - Nature et technologies) grant 299376 and NSERC Discovery grant 03267 [Guy Wolf]; NSF DMS grant 2327211 [Michael Perlmuter and Guy Wolf]. This research was also enabled in part by compute resources provided by Mila (mila.quebec). The content provided here is solely the responsibility of the authors and does not necessarily represent the official views of the funding agencies.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Alon, U. and Yahav, E. On the bottleneck of graph neural networks and its practical implications. In *International Conference on Learning Representations*, 2021.
- Amizadeh, S., Matushevych, S., and Weimer, M. Learning to solve circuit-SAT: An unsupervised differentiable approach. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=BJxgz2R9t7>.
- Amizadeh, S., Matushevych, S., and Weimer, M. PDP: A general neural framework for learning SAT solvers, 2020. URL <https://openreview.net/forum?id=S1xaf6VFPB>.
- Bao, L. L. N., Le, D. H., and Nguyen, D. A. Application of combinatorial optimization in logistics. In *2018 4th International Conference on Green Technology and Sustainable Development (GTSD)*, pp. 329–334. IEEE, 2018.
- Bello, I., Pham, H., Le, Q. V., Norouzi, M., and Bengio, S. Neural combinatorial optimization with reinforcement learning, 2017. URL <https://openreview.net/forum?id=rJY3vK9eg>.
- Berto, F., Hua, C., Zepeda, N. G., Hottung, A., Wouda, N., Lan, L., Park, J., Tierney, K., and Park, J. Routefinder: Towards foundation models for vehicle routing problems, 2024. URL <https://arxiv.org/abs/2406.15007>.
- Boisvert, L., Verhaeghe, H., and Cappart, Q. Towards a generic representation of combinatorial problems for learning-based approaches. In Dilkina, B. (ed.), *Integration of Constraint Programming, Artificial Intelligence, and Operations Research*, pp. 99–108, Cham, 2024. Springer Nature Switzerland.
- Chew, J., Hirn, M., Krishnaswamy, S., Needell, D., Perlmutter, M., Steach, H., Viswanath, S., and Wu, H.-T. Geometric scattering on measure spaces. *Applied and Computational Harmonic Analysis*, 70:101635, 2024.
- Cook, S. A. The complexity of theorem-proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, STOC ’71, pp. 151–158, New York, NY, USA, 1971. Association for Computing Machinery. ISBN 9781450374644. doi: 10.1145/800157.805047. URL <https://doi.org/10.1145/800157.805047>.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. *Introduction to Algorithms*. The MIT Press, Cambridge, Massachusetts, fourth edition, 2022. ISBN 9780262046305.
- Dai, H., Khalil, E. B., Zhang, Y., Dilkina, B., and Song, L. Learning combinatorial optimization algorithms over graphs. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS’17, pp. 6351–6361, Red Hook, NY, USA, 2017. Curran Associates Inc. ISBN 9781510860964.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. BERT: Pre-training of deep bidirectional transformers for language understanding. In Burstein, J., Doran, C., and Solorio, T. (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1423. URL <https://aclanthology.org/N19-1423/>.
- Drakulic, D., Michel, S., and Andreoli, J.-M. GOAL: A generalist combinatorial optimization agent learning, 2024. URL <https://arxiv.org/abs/2406.15079>.
- Eliwa, E. H. I. Enhancing skin cancer diagnosis through fine-tuning of pretrained models: A two-phase transfer learning approach. *International Journal of Breast Cancer*, 2025(1):4362941, 2025. doi: <https://doi.org/10.1155/ijbc/4362941>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1155/ijbc/4362941>.
- Feng, S., Sun, W., Li, S., Talwalkar, A., and Yang, Y. Frontierco: Real-world and large-scale evaluation of machine learning solvers for combinatorial optimization, 2026. URL <https://arxiv.org/abs/2505.16952>.
- Filar, J. A., Haythorpe, M., and Taylor, R. Linearly-growing reductions of Karp’s 21 NP-complete problems, 2019. URL <https://arxiv.org/abs/1902.10349>.
- Gama, F., Ribeiro, A., and Bruna, J. Diffusion scattering transforms on graphs. In *International Conference on Learning Representations*, 2018.
- Gao, F., Wolf, G., and Hirn, M. Geometric scattering for graph data analysis. In *International Conference on Machine Learning*, pp. 2122–2131. PMLR, 2019.
- Garey, M. R. and Johnson, D. S. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979. ISBN 0-7167-1044-7.
- Gasse, M., Chételat, D., Ferroni, N., Charlin, L., and Lodi, A. *Exact combinatorial optimization with graph convolutional neural networks*. Curran Associates Inc., Red Hook, NY, USA, 2019.
- Glover, F., Kochenberger, G., Hennig, R., and Du, Y. Quantum bridge analytics I: a tutorial on formulating and using QUBO models. *Annals of Operations Research*,

- 314(1):141–183, April 2022. ISSN 1572-9338. doi: 10.1007/s10479-022-04634-2. URL <http://dx.doi.org/10.1007/s10479-022-04634-2>.
- Guo, Z., Li, Y., Wang, S., and Yan, J. Learning general representations across graph combinatorial optimization problems, 2025. URL <https://openreview.net/forum?id=elmTU101oS>.
- Hendrycks, D. and Gimpel, K. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.
- Ioffe, S. and Szegedy, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In Bach, F. and Blei, D. (eds.), *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pp. 448–456, Lille, France, 07–09 Jul 2015. PMLR. URL <https://proceedings.mlr.press/v37/ioffel15.html>.
- Johnson, D. S. and Trick, M. A. (eds.). *Cliques, Coloring, and Satisfiability: Second DIMACS Implementation Challenge, October 11–13, 1993*, volume 26 of *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*. American Mathematical Society, Providence, RI, 1996. ISBN 9780821866092.
- Karalias, N. and Loukas, A. Erdos goes neural: an unsupervised learning framework for combinatorial optimization on graphs. *Advances in Neural Information Processing Systems*, 33:6659–6672, 2020.
- Karp, R. Reducibility among combinatorial problems. volume 40, pp. 85–103, 01 1972. ISBN 978-3-540-68274-5. doi: 10.1007/978-3-540-68279-0_8.
- Kaur, P. and Mahajan, P. Detection of brain tumors using a transfer learning-based optimized resnet152 model in MR images. *Computers in Biology and Medicine*, 188:109790, 2025. ISSN 0010-4825. doi: <https://doi.org/10.1016/j.combiomed.2025.109790>. URL <https://www.sciencedirect.com/science/article/pii/S0010482525001404>.
- Kaur, R., Kumar, R., and Gupta, M. Review on transfer learning for convolutional neural network. In *2021 3rd International Conference on Advances in Computing, Communication Control and Networking (ICAC3N)*, pp. 922–926, 2021. doi: 10.1109/ICAC3N53548.2021.9725474.
- Kochenberger, G., Hao, J.-K., Glover, F., Lewis, M., Lü, Z., Wang, H., and Wang, Y. The unconstrained binary quadratic programming problem: a survey. *J. Comb. Optim.*, 28(1):58–81, July 2014. ISSN 1382-6905. doi: 10.1007/s10878-014-9734-0. URL <https://doi.org/10.1007/s10878-014-9734-0>.
- Krylova, O. and Phillipson, F. Unsupervised learning with GNNs for QUBO-based combinatorial optimization. *EURO Journal on Computational Optimization*, 13:100116, 2025. ISSN 2192-4406. doi: <https://doi.org/10.1016/j.ejco.2025.100116>. URL <https://www.sciencedirect.com/science/article/pii/S2192440625000139>.
- Li, Y., Liu, J., Wu, Q., Qin, X., and Yan, J. Toward learning generalized cross-problem solving strategies for combinatorial optimization, 2025. URL <https://openreview.net/forum?id=VnaJNW80pN>.
- Li, Z., Chen, Q., and Koltun, V. Combinatorial optimization with graph convolutional networks and guided tree search. In *Neural Information Processing Systems*, 2018. URL <https://api.semanticscholar.org/CorpusID:53027872>.
- Lin, Z., Wu, Y., Zhou, B., Cao, Z., Song, W., Zhang, Y., and Jayavelu, S. Cross-problem learning for solving vehicle routing problems, 2024. URL <https://arxiv.org/abs/2404.11677>.
- Liu, F., Lin, X., Wang, Z., Zhang, Q., Tong, X., and Yuan, M. Multi-task learning for routing problem with cross-problem zero-shot generalization, 2024. URL <https://arxiv.org/abs/2402.16891>.
- Lucas, A. Ising formulations of many NP problems. *Frontiers in Physics*, Volume 2 - 2014, 2014. ISSN 2296-424X. doi: 10.3389/fphy.2014.00005. URL <https://www.frontiersin.org/journals/physics/articles/10.3389/fphy.2014.00005>.
- Min, Y., Wenkel, F., Perlmutter, M., and Wolf, G. Can hybrid geometric scattering networks help solve the maximum clique problem? *Advances in Neural Information Processing Systems*, 35:22713–22724, 2022.
- Min, Y., Bai, Y., and Gomes, C. P. Unsupervised learning for solving the travelling salesman problem. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=1AEc7aIW20>.
- Motzkin, T. S. and Straus, E. G. Maxima for graphs and a new proof of a theorem of turán. *Canadian Journal of Mathematics*, 17:533–540, 1965. doi: 10.4153/CJM-1965-053-6.
- Naseri, G. and Koffas, M. A. Application of combinatorial optimization strategies in synthetic biology. *Nature communications*, 11(1):2446, 2020.

- Oquab, M., Bottou, L., Laptev, I., and Sivic, J. Learning and transferring mid-level image representations using convolutional neural networks. In *2014 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1717–1724, 2014. doi: 10.1109/CVPR.2014.222.
- Papadimitriou, C. H. and Steiglitz, K. *Combinatorial optimization: algorithms and complexity*. Courier Corporation, 1998.
- Perlmutter, M., Tong, A., Gao, F., Wolf, G., and Hirn, M. Understanding graph neural networks with generalized geometric scattering transforms. *SIAM Journal on Mathematics of Data Science*, 5(4):873–898, 2023.
- Raghavan, P. Probabilistic construction of deterministic algorithms: Approximating packing integer programs. *Journal of Computer and System Sciences*, 37(2):130–143, 1988. ISSN 0022-0000. doi: [https://doi.org/10.1016/0022-0000\(88\)90003-7](https://doi.org/10.1016/0022-0000(88)90003-7). URL <https://www.sciencedirect.com/science/article/pii/0022000088900037>.
- Rampásek, L., Galkin, M., Dwivedi, V. P., Luu, A. T., Wolf, G., and Beaini, D. Recipe for a General, Powerful, Scalable Graph Transformer. *Advances in Neural Information Processing Systems*, 35, 2022.
- Sanokowski, S., Hochreiter, S., and Lehner, S. A diffusion model framework for unsupervised neural combinatorial optimization. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.
- Schuetz, M. J. A., Brubaker, J. K., and Katzgraber, H. G. Combinatorial optimization with physics-inspired graph neural networks. *Nature Machine Intelligence*, 4(4): 367–377, April 2022a. ISSN 2522-5839. doi: 10.1038/s42256-022-00468-6. URL <http://dx.doi.org/10.1038/s42256-022-00468-6>.
- Schuetz, M. J. A., Brubaker, J. K., Zhu, Z., and Katzgraber, H. G. Graph coloring with physics-inspired graph neural networks. *Phys. Rev. Res.*, 4: 043131, Nov 2022b. doi: 10.1103/PhysRevResearch.4.043131. URL <https://link.aps.org/doi/10.1103/PhysRevResearch.4.043131>.
- Selsam, D., Lamm, M., Bünz, B., Liang, P., de Moura, L., and Dill, D. L. Learning a SAT solver from single-bit supervision. In *International Conference on Learning Representations*, 2019. URL https://openreview.net/forum?id=HJMC_iA5tm.
- Sipser, M. *Introduction to the Theory of Computation*. International Thomson Publishing, 1st edition, 1996. ISBN 053494728X.
- Tönshoff, J., Ritzert, M., Wolf, H., and Grohe, M. Graph neural networks for maximum constraint satisfaction. *Frontiers in Artificial Intelligence*, Volume 3 - 2020, 2021. ISSN 2624-8212. doi: 10.3389/frai.2020.580607. URL <https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2020.580607>.
- Tönshoff, J., Kisin, B., Lindner, J., and Grohe, M. One model, any CSP: Graph neural networks as fast global search heuristics for constraint satisfaction. In Elkind, E. (ed.), *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI-23*, pp. 4280–4288. International Joint Conferences on Artificial Intelligence Organization, 8 2023. doi: 10.24963/ijcai.2023/476. URL <https://doi.org/10.24963/ijcai.2023/476>. Main Track.
- Wang, C., Fu, Z.-H., Lu, P., and Yu, T. Efficient training of multi-task neural solver for combinatorial optimization, 2025. URL <https://arxiv.org/abs/2305.06361>.
- Wenkel, F., Cantürk, S., Horoi, S., Perlmutter, M., and Wolf, G. Towards a general recipe for combinatorial optimization with multi-filter gnns. In Wolf, G. and Krishnaswamy, S. (eds.), *Proceedings of the Third Learning on Graphs Conference*, volume 269 of *Proceedings of Machine Learning Research*, pp. 3:1–3:20. PMLR, 26–29 Nov 2025. URL <https://proceedings.mlr.press/v269/wenkel25a.html>.
- Yolcu, E. and Póczos, B. Learning local search heuristics for boolean satisfiability. In Wallach, H., Larochelle, H., Beygelzimer, A., d’Alché-Buc, F., Fox, E., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/12e59a33de1bf0630f46edfe13d6ea2-Paper.pdf.
- You, K., Liu, Y., Wang, J., and Long, M. LogME: Practical assessment of pre-trained models for transfer learning. In Meila, M. and Zhang, T. (eds.), *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 12133–12143. PMLR, 18–24 Jul 2021. URL <https://proceedings.mlr.press/v139/you21b.html>.
- Zeiler, M. D. and Fergus, R. Visualizing and understanding convolutional networks. In Fleet, D., Pajdla, T., Schiele, B., and Tuytelaars, T. (eds.), *Computer Vision – ECCV 2014*, pp. 818–833, Cham, 2014. Springer International Publishing. ISBN 978-3-319-10590-1.

- Zeng, R., Lei, M., Niu, L., and Cheng, L. A unified pre-training and adaptation framework for combinatorial optimization on graphs, 2023. URL <https://arxiv.org/abs/2312.11547>.
- Zhang, D., Dai, H., Malkin, N., Courville, A., Bengio, Y., and Pan, L. Let the flows tell: Solving graph combinatorial optimization problems with gflownets. *arXiv preprint arXiv:2305.17010*, 2023.
- Zhong, L. and Tang, G. Preface: Combinatorial optimization drives the future of health care. *Journal of Combinatorial Optimization*, 42:675–676, 2021.
- Zhou, J., Cao, Z., Wu, Y., Song, W., Ma, Y., Zhang, J., and Xu, C. MVMoE: Multi-task vehicle routing solver with mixture-of-experts, 2024. URL <https://arxiv.org/abs/2405.01029>.
- Zong, Z., Wei, X., Zhang, G., Gao, C., Wang, H., and Li, Y. UniCO: Towards a unified model for combinatorial optimization problems, 2025. URL <https://arxiv.org/abs/2505.06290>.
- Zou, D. and Lerman, G. Graph convolutional neural networks via scattering. *Applied and Computational Harmonic Analysis*, 49(3):1046–1074, 2020.

A. Graph Combinatorial Optimization Problem Descriptions

We consider a graph $G = (V, E)$ with $|V| = N$. For problems defined on vertex subsets, we let $X \in \{0, 1\}^N$ denote a hard assignment vector where $X_i = 1$ indicates that v_i is in the set of interest. When training our network, we will approximate X_i by a soft assignment vector $\mathbf{p} \in [0, 1]^N$, where $\mathbf{p}_i$ is interpreted as the probability that $X_i = 1$. However, when defining the problems, it is more natural to use the hard assignment vector X .

MaxCut aims to find a partition $V' \sqcup V'' = V$ maximizing the number of cut edges. We minimize

$$- \sum_{(i,j) \in E} \frac{1 - \sigma_i \sigma_j}{2}$$

where $\sigma_i = 2X_i - 1$. Note that each term vanishes when v_i and v_j are in the same subset and decreases the loss when they are separated.

Maximum Clique aims to find the largest subset $V' \subseteq V$ such that all pairs of nodes in V' are connected. We minimize

$$-A \sum_{i=1}^N X_i + B \sum_{(i,j) \notin E} X_i X_j$$

where the first term favors large cliques, while the second penalizes missing edges inside V' .

Minimum Dominating Set aims to find the smallest subset $V' \subseteq V$ such that every node in $V \setminus V'$ has a neighbor in V' . We minimize

$$A \sum_{i=1}^N X_i + B \sum_{i=1}^N (1 - X_i) \prod_{v_j \in \mathcal{N}(v_i)} (1 - X_j)$$

where the first term favors small dominating sets and the second term penalizes nodes that are neither selected nor adjacent to any selected node (with $\mathcal{N}(v_i)$ denoting the immediate neighbors of v_i).

Maximum Independent Set aims to find the largest subset $V' \subseteq V$ such that no two nodes in V' are adjacent. We minimize

$$-A \sum_{i=1}^N X_i + B \sum_{(i,j) \in E} X_i X_j$$

where the first term maximizes the size of V' , while the second penalizes edges between nodes of V' .

Minimum Vertex Cover aims to find the smallest subset $V' \subseteq V$ such that every edge has at least one endpoint in V' . We minimize

$$A \sum_{i=1}^N X_i + B \sum_{(i,j) \in E} (1 - X_i)(1 - X_j)$$

where the first term minimizes the size of V' , and the second term penalizes uncovered edges.

Given K colors, **K-Coloring** aims to assign one color to each node so that all adjacent nodes have different colors. Here, we use an assignment matrix $X \in [0, 1]^{N \times K}$, where $X_{i,k} \approx 1$ indicates that node i has color k . We minimize

$$\sum_{i=1}^N \left(1 - \sum_{k=1}^K X_{i,k} \right)^2 + \sum_{(i,j) \in E} \sum_{k=1}^K X_{i,k} X_{j,k}$$

where the first term enforces one color per node, and the second penalizes monochromatic edges. We measure success of this tasks by the number for violations observed, i.e. the number of edges connecting two vertices of the same color is to be minimized.

B. Experimental Setting

Datasets. We conduct our experiments on the same synthetic datasets used in Wenkel et al. (2025), which are commonly used in the field of neural graph combinatorial optimization. In all experiments, we use datasets of size 6000 with 200-300 nodes (small) and 800-1200 nodes (large). In Section 4, we use graphs generated from the RB model. For RB graphs, we can specify the number of cliques (n) and the number of nodes per cliques (k). We use $n \in [20, 25]$ and $k \in [5, 12]$ for RB-small and $n \in [40, 55]$ and $k = 20$ for RB-large. We then use Barabási–Albert (BA) graphs in Section 5. For BA graphs, we can specify how many edges are attached from each new node, which we set to 4 for both BA-small and BA-large.

Backbone architecture. We use a GCON backbone based on Wenkel et al. (2025) with the specific configuration detailed in Table 8 for every experiment, except learning rate scheduling which varies depending on pretraining or fine-tuning. Total number of parameters depend on the size of the head used, but the GCON backbone always has 39.2K parameters.

Table 8. GCON Backbone and Training Hyperparameters

Parameter	Value
<i>Architecture Details</i>	
Number of GCON Layers	16
Hidden Dimension (d_h)	64
Channels (Filters)	[0], [1], [2], [4], [0,1], [1,2], [2,4]
Aggregation	Sum
Dropout Rate	0.3
Backbone Activation	GELU
Pre-MP / Post-MP Layers	1 / 1
Node Encoder Stats	Degree, Cluster Coeff., Triangles
<i>Optimization & Training</i>	
Optimizer	AdamW ($\eta = 10^{-3}$, $w_d = 0$)
Learning Rate Scheduler	Cosine with Warmup (50 epochs)
Batch Size	256 (small) or 128 (large)
Loss Strategy	Weighted Sum (Task weight = 1.0)

Decoders. The candidate sets are constructed by first ranking all vertices in descending order of their model-predicted scores. For each problem type, the algorithm generates K distinct candidate sets (seeds) in parallel to find the best possible solution for that specific graph. In the MaxClique and MIS decoders, the k -th candidate set is initialized with the k -th ranked node, and the algorithm then greedily attempts to add subsequent nodes in the ranked list only if they respect the problem constraint, such as forming a clique or an independent set. For the MDS and MVC, the k -th candidate set is constructed by ignoring the top $k - 1$ ranked nodes (by setting them to $-\infty$) and then greedily selecting the highest-ranked available nodes until the required coverage constraint is satisfied. MaxCut simply picks nodes with a score higher than 0.5 to be in the first set and the rest in the second and counts the number of cuts created by this partition, while coloring simply counts the violations (adjacent nodes of the same color).

C. Multi-Task Learning vs. Unified-Task Reductions

Table 9. Average number of nodes and edges for the original BA-small graph where we train on solving MDS, and the equivalent graphs for other tasks obtained by applying reductions to the original graphs. For the transformed graphs, we denote the $\times$ change in the “blow-up” in the nodes and edges of the resulting graph.

REDUCTION	# NODES	$\times$ MDS SIZE	# EDGES	$\times$ MDS SIZE
MDS (BASE)	250.3 $\pm$ 28.9	–	985.0 $\pm$ 115.8	–
MIS	1239.0 $\pm$ 146.8	4.95	2964.0 $\pm$ 352.3	3.01
MVC	1239.0 $\pm$ 146.8	4.95	2964.0 $\pm$ 352.3	3.01
MAXCLIQUE	30873.7 $\pm$ 7148.0	123.37	91870.3 $\pm$ 21357.4	93.27

A related but distinct set of approaches in neural CO instead focus on generalized formulations of CO problems in the form of *satisfiability problems* (SAT) or their optimization counterparts *Max-SAT*. This line of work emerges from the Cook-Levin

Table 10. Comparison of our MTL framework with unified-MDS-reduction-based approaches, where we train a GCON model to solve MDS on BA-small graphs, and then “transform” the graphs corresponding to MIS/MVC/MaxClique via constructing the true reductions over the graph. We test three variants of the pretrained MDS model: 0-shot, frozen backbone (F-BB), and full fine-tuning (FT). * denotes the original MDS pretraining result, which is trained for 100 epochs as opposed to constraining to 20 epochs for other tasks.

METHOD	MDS ↓	MVC ↓	MIS ↑	MAXCLIQUE ↑
(MDS RED.) RANDOM	120.43 ± 25.3	190.11 ± 36.9	59.87 ± 40.1	1.61 ± 1.04
(MDS RED.) BASE (20 EPOCHS)	31.37 ± 0.62*	165.21 ± 0.96	80.99 ± 6.42	2.98 ± 0.12
(MDS RED.) 0-SHOT	31.37 ± 0.62*	148.21 ± 2.53	102.78 ± 2.53	2.46 ± 0.44
(MDS RED.) F-BB	-	144.59 ± 1.35	106.25 ± 1.35	2.27 ± 0.14
(MDS RED.) FT	-	142.79 ± 1.37	106.45 ± 0.92	2.82 ± 0.01
(MTL) FT (W/ GRAPH STATS)	32.03 ± 0.66	139.31 ± 0.17	111.96 ± 0.02	4.34 ± 0.01
(MTL) FT (W/O GRAPH STATS)	36.57 ± 1.09	<u>140.14 ± 0.81</u>	<u>111.39 ± 0.39</u>	<u>4.31 ± 0.01</u>

theorem (Cook, 1971; Karp, 1972), which shows that any problem in NP can be reduced to a boolean satisfiability problem (SAT) in polynomial time.

Zeng et al. (2023) pretrain GNNs on bipartite graph representations of synthetic Max-SAT instances in a supervised manner and solve MaxCut, MIS and MDS instances by reducing them to Max-SAT graphs. Guo et al. (2025) follow a similar recipe but focus on SAT formulations of corresponding decision problems, and pretrain in an unsupervised manner via contrastive learning. These works acknowledge the value of generalization for neural CO and provide partial solutions, but do not directly address the relationship between reducibility and transferability; instead, they aim to wholly *circumvent* task transferability itself by directly learning in the general SAT/Max-SAT space, to which all our CO problems of interest can be polynomially reduced.

These approaches, however, give rise to different limitations on transferability over *graph* distributions, rather than over tasks. Specifically, the approaches above require applying the true reduction over the graph of interest, similar to the MIS $\leftrightarrow$ MC reduction (Sec. 4.2) where we demonstrate that finding the maximum clique of G is equivalent to finding the MIS of the complement $\bar{G}$. The graph reductions between MIS, MVC and MaxClique, however, lead to relatively *mild* structural changes, as the node set V is maintained. As discussed in Sec.4, even such mild changes in structure lead to tangible limitations on transferability in constrained fine-tuning settings.

The reductions involved to map an, e.g., MaxClique graph to an equivalent MDS or Max-SAT graph, on the other hand, may require “chaining” multiple structure-altering reductions to arrive at the MDS/Max-SAT representations, which in turn leads to extreme blow-ups in the number of nodes and edges considered. For example, reducing MaxCut on a graph to its SAT counterpart, in practice, requires chaining three reductions: MaxCut $\rightarrow$ MIS $\rightarrow$ 3-SAT $\rightarrow$ SAT.

This resulting blow-up, despite being polynomially bounded, leads to a large distribution shift in the graph distributions across which the models are trained and transferred to, as well as substantially larger graphs in general, limiting the ability of GNN-based frameworks to learn the appropriate graph representations effectively. To demonstrate the potential shortcomings of these methods compared to our multi-task learning (MTL) framework, we hereby present an analogous study where we compare our MTL framework with a “unified-task” learning framework akin to the works discussed above.

Unified-task method. We pretrain a GCON model to solve MDS on BA-small graphs, and evaluate the transferability on the resulting model on the *reduced* versions of the BA-small graphs for MIS, MVC and MaxClique problems, respectively. The reducibility relationships are defined as

$$\text{MDS} \leftarrow \text{MVC} \leftrightarrow \text{MIS} \leftrightarrow \text{MaxClique}$$

where each arrow indicates whether a known reduction exists in the specified direction. We choose MDS as our “base” task as all other tasks can be reduced to MDS, whereas no established direct reduction from MDS to the other tasks exists to our knowledge without reducing it to a more general CO framing (e.g. Max-SAT). In addition to the MVC $\leftrightarrow$ MIS and MIS $\leftrightarrow$ MaxClique reductions we establish in Sec.4, we introduce the following reduction (Sipser, 1996):

The MVC on G is thus equivalent to the MDS on G' . The chaining of reductions, in the case of MaxClique $\rightarrow$ MDS, is then

Algorithm 1 $\text{MDS} \leftarrow \text{MVC}$

Require: Undirected graph $G := (V, E)$

$G' := (V', E') \leftarrow G$

Remove all nodes with no incident edges from G'

for $(i, j) \in E$ **do**

$V' \leftarrow V' \cup \{k\}$ {For every original edge (i, j) , add a new “gadget” node k to V' }

$E' \leftarrow E' \cup \{(i, k), (j, k)\}$ {Connect the new node k to both endpoints of the original edge}

end for

return G'

as follows:

1. Convert the original graph $G_{\text{MaxClique}}$ to its complement to define the equivalent MIS graph $G_{\text{MIS}} = G_{\text{MVC}}$.
2. Apply the reduction denoted in Algorithm 1 to obtain the MDS graph G_{MDS} .
3. Solve the MDS problem on G_{MDS} .
4. Invert the solution set to account for the $\text{MVC} \leftrightarrow \text{MIS}$ reduction (which does not transform the graph but inverts the node assignments). The resulting set (and thus its cardinality) is equivalent to $G_{\text{MaxClique}}$.

We follow this recipe to generate reduced versions of our BA-small dataset for MIS, MVC and MaxClique respectively. The node and edge counts of each reduced dataset is presented in Table 9: The first graph reduction from MDS to MIS/MVC leads to graphs with five times the number of nodes and three times the number of edges. The chained reduction to MaxClique leads to an average graph size of about 31,000 nodes, a 122-fold increase over the relatively small (200-300-node) original graphs.

We then evaluate our MDS pretrained model on each reduced dataset in three transfer settings: zero-shot transfer, fine-tuning the linear prediction head while keeping the backbone frozen, and fine-tuning the whole model. In line with our MTL experiments, we restrict fine-tuning to 20 epochs for a fair comparison.

Results. We present the results of our study in Table 10. Our MTL method consistently outperforms all variants of the unified-MDS-task model, with the performance gap growing as we chain more reductions and increase the distribution shift. For MVC and MIS, unified-task pretraining is still useful despite falling short of MTL performance, with noticeable gains even in the zero-shot setting compared to the baseline of training 20 epochs from scratch, with additional improvements through fine-tuning.

However, in the MaxClique case with a $\sim 100\times$ blow-up in the node and edge counts, unified-task pretraining breaks down: Even in the full fine-tuning setting, the attained maximum clique is lower than the baseline, indicating negative transfer. The gap between fine-tuned unified-task and MTL methods also increase from 2.5% and 4.9% for MVC and MIS, respectively, to 35.0% for MaxClique.

Analysis. These results suggest a direct inverse correlation between increased distribution shift as a result of chained reductions, and reduced transferability. In addition to the distribution shift itself, the induced large graphs also limit the effectiveness of fine-tuning by introducing practical and computational bottlenecks that reduce the quality of learned graph representations. The practical bottlenecks are straightforward to analyze: Significantly larger (and denser in cases where the $\text{MIS} \leftrightarrow \text{MaxClique}$ reduction, which returns dense complements of sparse graphs, is considered) graphs render efficient training of GNNs expensive, require significantly more memory and compute, and provide noisier gradients due to being limited to small batch sizes. Computing graph statistics or positional encodings (PE) for graph datasets with sizes in the order of tens of thousands also render their usage as informative node features cumbersome at best, and non-viable at worst. We emphasize that we have considered BA-small graphs precisely for these reasons: Despite the small size of the original graphs, the resulting MaxClique dataset is comparatively so large that when training on a single NVIDIA L40S GPU, we can fit only four graphs into memory at a time and full fine-tuning (20 epochs) takes about two days, whereas training the MDS model on the original BA-small dataset only takes several hours.

Learning non-local relationships on large graphs are also more prone to run into under-reaching and over-squashing (Alon & Yahav, 2021), well-known challenges in graph representation learning that limit the expressivity of the learned graph representations. Without auxiliary structures like virtual nodes, skip connections or graph Transformer layers, we conjecture

Table 11. Fine-tuning on MDS-MIS-Coloring pretrained backbones for 20 epochs (BA-large dataset, 3 runs average)

PROBLEM	FINE-TUNED	BASELINE	FULL
↑ MAXCUT	2935.82 ± 0.63	2926.39 ± 2.46	2946.01 ± 6.00
↑ MAXCLIQUE	4.31 ± 0.03	4.32 ± 0.01	4.41 ± 0.02
↓ MDS	109.72 ± 0.16	129.64 ± 12.74	121.20 ± 10.38
↑ MIS	453.51 ± 0.17	452.59 ± 0.07	454.77 ± 0.13
↓ MVC	550.55 ± 0.36	554.53 ± 1.22	548.86 ± 0.35
↓ COLORING	31.40 ± 5.29	53.48 ± 4.53	10.61 ± 2.95

Table 12. Size Extrapolation Comparison on BA-large and BA-small datasets (3 runs average)

PROBLEM	BA-LARGE				BA-SMALL			
	FINE-TUNED	EXTRAPOL.	DIFF	BASELINE	FINE-TUNED	EXTRAPOL.	DIFF	BASELINE
↑ MAXCUT	2935.8 ± 0.6	2936.0 ± 0.1	+0.01%	2926.4 ± 2.5	722.8 ± 1.0	722.0 ± 1.4	-0.11%	716.9 ± 1.5
↑ MAXCLIQUE	4.31 ± 0.03	4.30 ± 0.02	-0.24%	4.32 ± 0.01	4.32 ± 0.01	4.34 ± 0.01	+0.49%	4.31 ± 0.00
↓ MDS	109.7 ± 0.2	111.9 ± 0.8	+2.01%	129.6 ± 12.7	29.6 ± 0.1	29.2 ± 0.1	-1.61%	33.9 ± 1.2
↑ MIS	453.5 ± 0.2	453.9 ± 0.2	+0.08%	452.6 ± 0.1	111.9 ± 0.1	111.8 ± 0.0	-0.10%	111.3 ± 0.3
↓ MVC	550.5 ± 0.4	550.9 ± 0.3	+0.06%	554.5 ± 1.2	139.7 ± 0.3	139.8 ± 0.2	+0.09%	141.4 ± 0.2
↓ COLORING	31.4 ± 5.3	53.8 ± 12.5	+71.26%	53.5 ± 4.5	17.3 ± 2.0	28.3 ± 8.8	+63.52%	49.0 ± 21.8

that it is difficult for conventional GNNs to fully overcome these bottlenecks both when training from scratch and in fine-tuning, leading to both the baseline and unified-task models falling well short of our MTL model which instead solves the respective tasks on the original graph.

D. Additional Experiments

Multi-task learning on large graphs. We test our pretraining-finetuning framework developed in Section 5 on BA-large graphs. We see that the results obtained and the observations made in Section 5 also hold for larger graphs.

Size extrapolation. We investigate if finetuning on graphs of a certain size allows us to quickly finetune to graphs of different sizes never seen in pretraining. Using the same set-up as Section 5, we pretrain on BA-small graphs and finetune on BA-large graphs, and vice-versa. We observe very good size extrapolation, where for most tasks pretraining on small graphs then finetuning on larger graphs and vice-versa performs as good as pretraining and finetuning on graphs of the same size. Only coloring doesn’t transfer as well, where we see that extrapolating from large to small graphs gives better results than training from scratch but not nearly as good as pretraining from same size graphs, while extrapolating from small to large graphs gives similar results to training from scratch.

Timing experiments. We record the required time for inference at test time once we have pretrained our model as done in Sec. 5 to finetune on our 6 different CO problems. We use one NVIDIA L40S GPU for each task for fair comparison. Pretraining on single tasks take from a few minutes (MaxCut, Coloring) to a few hours (backbone, MDS, MaxClique).

Table 13. Inference Time Performance on NVIDIA L40S GPU (seconds)

PROBLEM	BA-LARGE	BA-SMALL
MAXCUT	6.23	2.23
MAXCLIQUE	48.95	32.58
MDS	46.42	10.65
MIS	65.11	22.78
MVC	137.52	31.39
COLORING	6.38	2.84

We also provide below the inference times for Table 2 over the test set (500 RB-small graphs, NVIDIA Tesla V100). Note that despite comparable times to the other methods on the RB-small dataset, GFN and Gurobi scale substantially worse

compared to the GNN-based methods on larger graphs. MVC times for Gurobi and GFN are not listed, but would be approximately equal to MIS as one would solve for MIS and invert the solution.

Table 14. Inference times of Table 2 over the RB-small test set (500 graphs, NVIDIA Tesla V100).

Method	Type	MVC Time	MClique Time	MIS Time
True Size	—	—	—	—
Gurobi	OR	—	1:55	47:34
GFN	SSL	—	0:42	0:32
GCN	SSL-GNN	1:29	0:57	0:45
GIN	SSL-GNN	1:24	0:55	0:35
GATv2	SSL-GNN	1:26	1:02	0:57
GCON	SSL-GNN	1:46	1:20	1:05

Zero-shot performance on DIMACS instances. We investigate how pretraining using our method allows us to extrapolate to new graph distribution by evaluating on DIMACS instances, which are a standard reference point for many combinatorial optimization problems. As DIMACS instances consist of specific graph instances and not a standard graph dataset adaptable to our framework, we report inference-only, zero-shot transfer results by running our pretrained model on select DIMACS instances (treating them as a test set) to demonstrate that the learned representations produce reasonable solutions on out-of-distribution real-world graphs. Using the evaluation framework inspired from [Feng et al. \(2026\)](#), we do this for the MIS task by finding the MIS of the complement of DIMACS MaxClique graphs, which is equivalent to the MaxClique of the original DIMACS graphs for which the best known sizes are available (2nd DIMACS Challenge, [Johnson & Trick \(1996\)](#)). Comparing our pretrained model to random initialization, we observed that it comfortably outperforms random models on almost all graphs, and obtains reasonable zero-shot results on many instances, showing that we are able to extrapolate useful information from pretraining on RB-graphs to considerably out-of-distribution hard graph instances. Note we are not expected to attain near-SOTA results when zero-shot learning on considerably out-of-distribution hard single graph instances using an ML-based solver, compared to SOTA algorithmic solvers specifically designed for MIS.

Table 15. Comparison of our pretrained (RB small) and a randomly initialized model on 0-shot inference on DIMACS instances for MIS.

Graph name	Pretrained	Random	Best known
C1000.9	56	29	68
C125.9	32	25	34
C2000.5	11	9	16
C2000.9	48	28	80
C250.9	36	28	44
C400.5	10	8	18
C500.9	50	30	57
DSJC1000.5	11	8	15
DSJC500.5	10	8	13
MANN_a27	117	85	126
MANN_a45	276	71	345
MANN_a81	300	122	1100
brock200.2	10	8	12
brock200.4	13	11	17
brock400.2	21	16	29
brock400.4	23	15	33
brock800.2	16	12	24
brock800.4	16	12	26
gen200_p0.9_44	37	28	44
gen200_p0.9_55	38	30	55
gen400_p0.9_55	48	30	55
gen400_p0.9_65	44	31	65
gen400_p0.9_75	45	33	75
hamming10-4	27	16	40
hamming8-4	16	16	16
keller4	9	8	11
keller5	18	15	27
keller6	37	28	59
p_hat1500-1	10	3	12
p_hat1500-2	52	4	65
p_hat1500-3	79	10	94
p_hat300-1	7	5	8
p_hat300-2	24	10	25
p_hat300-3	31	17	36
p_hat700-1	8	3	11
p_hat700-2	38	5	44
p_hat700-3	57	10	62